

UNIVERSIDAD AUTONOMA DE SINALOA

UNIDAD ACADEMICA FACULTAD DE INGENIERIA MOCHIS

LICENCIATURA EN INGENIERIA DE SOFTWARE



ASIGNATURA:

Administración de Sistemas

DOCENTE:

Dr. Herman Geovanny Ayala Zuñiga

ALUMNO:

Alberto Torres Chaparro

GRUPO:

301

TEMA:

Dockers

Introduccion y Arquitectura

Objetivo

Migrar los servicios esenciales de practicas anteriores (instalaciones locales) a contenedores Docker personalizados, automatizando su despliegue mediante Dockerfiles y docker-compose, con enfoque en seguridad, aislamiento de red y almacenamiento persistente.

Servicios Implementados

- Servidor web Nginx Alpine personalizado (imagen propia, usuario no-root, server_tokens off)
- Base de datos PostgreSQL 16 Alpine con volumen persistente db_data
- Contenedor de respaldo automatizado de PostgreSQL cada 5 minutos
- Servidor FTP con volumen compartido web_content para carga de archivos

Diagrama de Topologia

La siguiente tabla describe la topologia de red de la practica y como interactuan los contenedores dentro de la red bridge infra_red:

CLIENTE / HOST	-->	RED: infra_red 172.20.0.0/16	-->	CONTENEDORES
Contenedor	IP Fija	Puerto	Imagen Base	Volumen
web_principal	172.20.0.10	8080:8080	nginx:alpine (custom)	web_content
postgres_db	172.20.0.20	5432 (interno)	postgres:16-alpine	db_data
postgres_backup	172.20.0.40	-	postgres:16-alpine	./db/backups
ftp_archivos	172.20.0.30	21, 21000-21010	delfer/alpine-ftp-server	web_content

Nota: web_content es un volumen compartido entre web_principal y ftp_archivos, lo que permite que los archivos subidos por FTP sean inmediatamente servidos por Nginx.

Guia de Uso de los Scripts

Requisitos Previos

Requisito	Detalle
Sistema Operativo	Linux (RHEL, Fedora, Ubuntu) con bash 5+
Docker Engine	Version 24+ instalado y activo (systemctl status docker)
Docker Compose	Plugin v2 incluido con Docker Desktop o instalado por separado
Permisos	Ejecutar como root o usuario con privilegios sudo
Conexion a internet	Necesaria la primera ejecucion para descargar imagenes Docker
IP del host	Configurar variable HOST_IP si la IP del host no es 192.168.10.10

Instrucciones de Ejecucion

Paso 1: Dar permisos de ejecucion a los scripts

```
chmod +x main.sh 01_deploy_practical0.sh 02_pruebas_practical0.sh  
03_resumen_evidencias.sh 04_cleanup_practical0.sh
```

Paso 2: Ejecutar el orquestador principal

```
sudo bash main.sh
```

Si la IP del host es diferente a 192.168.10.10, exportar la variable antes de ejecutar: `export HOST_IP=<tu-ip> && sudo -E bash main.sh`

Flujo de Interaccion del Menu Principal

Opcion	Modulo	Descripcion
[1]	Despliegue completo	Genera todos los archivos de configuracion (docker-compose.yml, Dockerfile, nginx.conf, HTML) y levanta los 4 servicios con docker compose up -d --build
[2]	Ejecutar pruebas	Ejecuta el protocolo completo de validacion: persistencia BD, ping entre contenedores, subida FTP, y estadisticas de recursos
[3]	Resumen de evidencias	Muestra un reporte formateado con checklist de todos los requisitos de la practica. Ideal para tomar capturas de pantalla para la entrega
[4]	Limpieza	Solicita confirmacion y ejecuta docker compose down. Los volumenes db_data y web_content se conservan por defecto

[5]	Pipeline completo	Encadena automaticamente Despliegue -> Pruebas -> Evidencias en una sola ejecucion sin intervencion manual
[0]	Salir	Termina el programa

Datos que Solicita el Script

- Opcion 4 (Limpieza): solicita confirmacion 's' o 'N' antes de eliminar contenedores
- No se requieren datos adicionales: todas las IPs, puertos, credenciales y nombres estan preconfigurados en el script de despliegue
- Para personalizar la IP del host: exportar HOST_IP antes de ejecutar (ver Paso 2)

Bitacora de Desarrollo y Configuracion

Logica de los Scripts

main.sh - Orquestador

Actua como punto de entrada unico. Al iniciarse verifica que los 4 scripts modulo existan en el mismo directorio. Presenta un menu interactivo con 6 opciones y ejecuta cada modulo mediante la funcion `run_script()`, que invoca `bash` sobre la ruta absoluta del script seleccionado. La opcion Pipeline completo encadena los modulos 1, 2 y 3 secuencialmente sin intervencion del usuario.

01_deploy_practica10.sh - Despliegue

Genera en tiempo de ejecucion todos los archivos necesarios: `docker-compose.yml` con los 4 servicios y sus restricciones de recursos, el `Dockerfile` del servidor web, `nginx.conf` con `server_tokens` off, y el contenido HTML/CSS/SVG de la pagina. Utiliza heredocs (`<<EOF`) para escribir cada archivo desde `bash`, evitando dependencias externas. Finalmente ejecuta `docker compose up -d --build` para construir la imagen personalizada y levantar los servicios.

02_pruebas_practica10.sh - Protocolo de Pruebas

Ejecuta las 4 pruebas del protocolo de la practica. Mantiene dos contadores (PASS y FAIL) que se incrementan segun el resultado de cada validacion. Cada prueba usa comandos `docker exec` para interactuar con los contenedores en ejecucion. El resultado de cada prueba se evalua con condicionales `if` y se muestra un resumen al final con la proporcion de pruebas aprobadas.

03_resumen_evidencias.sh - Evidencias

Consulta el estado real del sistema mediante 7 bloques de evidencia, encuadrando la salida de cada comando dentro de bloques visuales. Al final genera un checklist automatico que evalua cada requisito de la practica usando test condicionales de `bash`, mostrando [OK] o [FALLO] por cada punto.

04_cleanup_practica10.sh - Limpieza

Solicita confirmacion interactiva antes de proceder. Ejecuta `docker compose down` para detener y eliminar los contenedores. Por diseno, los volúmenes `db_data` y `web_content` NO se eliminan, preservando la evidencia de persistencia requerida por la prueba 10.1.

Configuracion del Dockerfile (Servidor Web)

El `Dockerfile` del servidor web implementa varias practicas de seguridad:

- Imagen base `nginx:alpine` para minimizar la superficie de ataque
 - Se crea un grupo `webgroup` y usuario `webuser` sin privilegios mediante `addgroup/adduser` de Alpine
 - Los directorios temporales de Nginx se crean manualmente con los permisos correctos para el usuario `no-root`
 - La directiva `USER webuser` garantiza que el proceso Nginx nunca corre como `root`
 - La directiva `server_tokens off` en `nginx.conf` oculta la version del servidor en los headers HTTP
-

Configuracion de Red y Recursos

La red infra_red se configura como bridge con subnet 172.20.0.0/16. Cada contenedor recibe una IP estatica (ipv4_address), lo que permite que se comuniquen entre si por nombre de servicio (DNS interno de Docker Compose) y tambien por IP fija. Los limites de memoria y CPU se establecen mediante mem_limit y cpus en docker-compose.yml, usando el driver de control de grupos del kernel Linux (cgroups).

Pruebas de Funcionamiento (Validacion)

Protocolo de Pruebas

La siguiente tabla resume las 4 pruebas principales del protocolo de validacion de la practica:

Prueba	Entrada / Accion	Salida Esperada	Salida Obtenida
10.1	Crear tabla alumnos, insertar registro, ejecutar docker rm -f postgres_db, recrear contenedor	El registro 'altorcha' persiste en la BD despues de destruir y recrear el contenedor	Verificar con SELECT * FROM alumnos; - debe retornar el registro original
10.2	Ejecutar docker exec web_principal ping -c 4 db	4 paquetes transmitidos, 0% packet loss. El contenedor web resuelve el nombre 'db' por DNS interno	Salida de ping con 4 respuestas exitosas de 172.20.0.20
10.3	Subir archivo prueba_ftp.txt al servidor FTP via curlimages/curl con modo pasivo	El archivo aparece en /usr/share/nginx/html/uploads/ del contenedor web y es accesible via HTTP	ls -lh muestra prueba_ftp.txt. Accesible en http://<IP>:8080/uploads/prueba_ftp.txt
10.4	Ejecutar docker stats --no-stream	Todos los contenedores muestran uso de memoria dentro de sus limites configurados (512m / 256m)	Tabla de stats con columnas MEM USAGE / LIMIT y CPU % para los 4 contenedores

Conclusiones y Referencias

Lecciones Aprendidas

Dificultad 1: Nginx no-root y directorios temporales

Al cambiar el proceso de Nginx a un usuario sin privilegios, el servicio fallaba al iniciarse porque intentaba escribir en `/var/run/nginx.pid` y en los directorios de cache. La solución fue crear manualmente los directorios de cache (`/var/cache/nginx/*`) con `chown` al usuario `webuser` dentro del `Dockerfile`, y redirigir el `pid` a `/tmp/nginx.pid` en `nginx.conf`.

Dificultad 2: Volumen compartido FTP/Web con SELinux

En sistemas con SELinux activo (RHEL/Fedora), el montaje de volúmenes en rutas del host requiere la etiqueta `:Z` en la definición del volumen para que el contenedor tenga acceso de escritura. Esto se aplica específicamente al volumen de backups (`./db/backups:/backups:Z`). Sin la etiqueta, el contenedor de respaldo no podía escribir en la carpeta del host.

Dificultad 3: FTP pasivo y HOST_IP

El servidor FTP en modo pasivo requiere que la imagen conozca la IP pública/externa del host para incluirla en las respuestas PASV. Si se usa una IP incorrecta, el cliente FTP puede conectar al puerto 21 pero falla al transferir datos. La variable de entorno `ADDRESS` del contenedor `delfer/alpine-ftp-server` resuelve esto; se parametrizó como `HOST_IP` para facilitar el despliegue en diferentes entornos.

Dificultad 4: Tiempo de inicio de PostgreSQL

Al recrear el contenedor `postgres_db` en la prueba 10.1, el script debía esperar a que PostgreSQL terminara su proceso de recuperación del WAL antes de aceptar conexiones. Se implementó un bucle de espera de 10 segundos con retroalimentación visual en el terminal para garantizar que el `psql` no fallara por conexión rechazada.

Referencias

Fuente	Referencia
Docker Docs (oficial)	docs.docker.com - Dockerfile reference, Compose file reference, Networking overview, Use volumes
Nginx Docs (oficial)	nginx.org/en/docs/ - Core module: server_tokens directive, ngx_core_module
PostgreSQL Docs (oficial)	postgresql.org/docs/16/ - pg_dumpall, Client Authentication, Connection Settings
Man Pages Linux	man bash, man docker, man docker-compose, man nginx, man psql (disponibles via 'man <comando>')
delfer/alpine-ftp-server	github.com/delfer/docker-alpine-ftp-server - Variables de entorno USERS, ADDRESS, MIN_PORT, MAX_PORT
Anthropic Claude (IA)	claude.ai - Consultas sobre: estructura de heredocs en bash, configuracion de nginx no-root, logica de pruebas de persistencia Docker, formato de tablas en docx-js. Modelo: Claude Sonnet 4.5
OWASP Docker Security	cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html - no-new-privileges, cap_drop, usuario no-root
